Atelier pratique : OAuth 2.0 et OpenID Connect

Sécurisation d'une application React + Spring Boot avec Keycloak

Module : Sécurité des systèmes distribués

Objectif pédagogique

Déployer un serveur d'identité Keycloak ; authentifier une application React avec OpenID Connect et le flux Authorization Code avec PKCE ; protéger les API Spring Boot avec des Access Tokens ; appliquer des autorisations selon les rôles USER et ADMIN.

1 Rappels théoriques

Authentification
Vérifier l'identité d'un utilisateur : « Qui êtes-vous ? »

Autorisation
Vérifier les opérations que l'utilisateur a le droit d'effectuer.

OAuth 2.0
Framework d'autorisation permettant à une application d'obtenir un jeton donnant accès à une ressource protégée.

OpenID Connect (OIDC)
Couche d'authentification construite au-dessus d'OAuth 2.0. Elle introduit notamment l'ID Token.

Keycloak
Serveur d'identité permettant de centraliser l'authentification, la gestion des utilisateurs et des autorisations.

Access Token
Jeton utilisé pour accéder à une API protégée. Il est envoyé dans l'en-tête HTTP : `Authorization: Bearer <token>`. Il peut être au format JWT.

ID Token
Jeton OIDC contenant des informations relatives à l'authentification de l'utilisateur. Il n'est pas destiné à autoriser les appels à notre API.

Refresh Token
Jeton éventuellement délivré pour renouveler un Access Token sans demander à l'utilisateur de se reconnecter.

Realm
Espace de configuration Keycloak isolant les utilisateurs, les clients et les rôles.

Client
Application enregistrée auprès du serveur d'identité. Dans cet atelier, il s'agit du frontend React.

Resource Server
Serveur hébergeant les ressources protégées et validant les Access Tokens. Ici, il s'agit de Spring Boot.

PKCE
Proof Key for Code Exchange. Mécanisme qui sécurise l'échange du code d'autorisation contre des jetons. Il est particulièrement important pour les applications React exécutées dans un navigateur.

Attention

JWT est un format de jeton, OAuth 2.0 est un framework d'autorisation, et OpenID Connect est une couche d'authentification. Ces technologies sont complémentaires.

2 Étude de cas et architecture

Nous allons développer une application de **gestion des demandes**.
Deux types d'utilisateurs sont définis :

— **USER** : consulter ses demandes et créer une demande.

— **ADMIN** : consulter toutes les demandes et modifier leur statut.

Pour simplifier l'atelier, les demandes seront conservées en mémoire dans le backend. Elles disparaîtront au redémarrage de Spring Boot.

2.1 Architecture technique

Composant	Technologie	Port
Serveur d'identité	Keycloak	8080
API sécurisée	Spring Boot	8081
Frontend	React / Vite	5173

2.2 Fonctionnement global

1. L'utilisateur ouvre l'application React.
2. React redirige l'utilisateur vers Keycloak.
3. Keycloak authentifie l'utilisateur.
4. Keycloak retourne un code d'autorisation.
5. Le client échange ce code avec PKCE contre des jetons.
6. React transmet l'Access Token à Spring Boot.
7. Spring Security vérifie le JWT et les autorisations.
8. L'API retourne les ressources autorisées.

3 Prérequis

— Docker Desktop

— Java 17 ou 21

— Maven ou Maven Wrapper

— Node.js compatible avec Vite et npm

— Visual Studio Code ou IntelliJ IDEA

— Navigateur web

— Postman pour les tests

Toutes les applications seront exécutées localement.

4 Séance 1 : Keycloak et Spring Boot

4.1 Installation de Keycloak avec Docker

Créer un dossier nommé `atelier-oauth2`.
Dans ce dossier, créer le fichier suivant.

Fichier : `docker-compose.yml`

```
services:
  keycloak:
    image: quay.io/keycloak/keycloak:26.8.1
    container_name: keycloak-atelier
    command: start-dev
    environment:
      KC_BOOTSTRAP_ADMIN_USERNAME: admin
      KC_BOOTSTRAP_ADMIN_PASSWORD: admin
    ports:
      - "127.0.0.1:8080:8080"
    volumes:
      - keycloak_data:/opt/keycloak/data

volumes:
  keycloak_data:
```

Démarrer Keycloak :

```
docker compose up -d
```

Afficher les journaux :

```
docker compose logs -f keycloak
```

Ouvrir le navigateur :

http://localhost:8080

Accéder à l'interface d'administration avec :

— Username : `admin`

— Password : `admin`

Attention

Ces identifiants sont uniquement destinés au TP local. Le mode `start-dev` n'est pas destiné à la production.

4.2 Création d'un Realm

Dans Keycloak :

1. Ouvrir le sélecteur de Realm.
2. Cliquer sur **Create realm**.
3. Saisir `enset-realm`.
4. Valider.

Le Realm représente un espace indépendant contenant les utilisateurs, les rôles et les applications clientes.

4.3 Création du Client React

Dans le Realm `enset-realm` :

1. Accéder au menu **Clients**.
2. Cliquer sur **Create client**.
3. Choisir **OpenID Connect**.
4. Client ID : `react-client`.

5. Client authentication : **Off**.
6. Standard flow : **On**.
7. Implicit flow : **Off**.
8. Direct access grants : **Off**.

Configurer les URLs :

```
Root URL:
http://localhost:5173/

Home URL:
http://localhost:5173/

Valid redirect URIs:
http://localhost:5173/*

Web origins:
http://localhost:5173
```

Enregistrer la configuration.
Si l'option PKCE est disponible, sélectionner :

```
PKCE Code Challenge Method: S256
```

Explication : React est un client public, car le code JavaScript exécuté dans le navigateur ne peut pas conserver un secret confidentiel.

4.4 Création des rôles

Dans **Realm roles**, créer :

— `USER`

— `ADMIN`

Ces rôles permettent de contrôler les opérations accessibles dans l'application.

4.5 Création des utilisateurs

Dans le menu **Users**, créer deux utilisateurs.

Utilisateur	Mot de passe	Rôles
alice	alice123	USER
bob	bob123	USER, ADMIN

Pour chaque utilisateur :

1. Créer l'utilisateur dans **Users**.
2. Ouvrir l'onglet **Credentials**.
3. Définir le mot de passe.
4. Désactiver **Temporary**.
5. Ouvrir **Role mapping**.
6. Associer les rôles nécessaires.

Attention

Les rôles utilisés ici sont des **Realm roles**, et non des Client roles.
Ils sont normalement présents dans le JWT sous le claim `realm_access.roles` lorsque les mappers de rôles par défaut de Keycloak sont actifs.

5 Création du projet Spring Boot

Accéder à :

https://start.spring.io

Configurer le projet :

— Project : Maven
— Language : Java
— Spring Boot : version 3.5.x stable
— Group : `ma.formation`
— Artifact : `demandes-api`
— Java : 17

Ajouter les dépendances :

— Spring Web
— Spring Security
— OAuth2 Resource Server

Télécharger puis ouvrir le projet.

5.1 Configuration de l'application

Fichier : `src/main/resources/application.properties`

```
server.port=8081

spring.security.oauth2.resourceserver.jwt.issuer-uri=http://localhost:8080/realms/enset-realm
```

Explication :
La propriété `issuer-uri` indique l'émetteur des jetons JWT.
Spring Security utilise les métadonnées du Realm Keycloak pour découvrir les clés publiques nécessaires à la validation des signatures.
Il vérifie notamment la signature, l'émetteur et la validité temporelle du jeton.

6 Configuration de Spring Security

Créer la classe suivante dans le package `ma.formation.demandesapi`.

Fichier : `SecurityConfig.java`

```
package ma.formation.demandesapi;

import java.util.ArrayList;
import java.util.Collection;
import java.util.List;
import java.util.Map;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.oauth2.server.resource.authentication.JwtAuthenticationConverter;
```

```
import org.springframework.security.oauth2.server.resource.authentication.
    JwtGrantedAuthoritiesConverter;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

@Configuration
public class SecurityConfig {

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http)
            throws Exception {

        return http
            .csrf(csrf -> csrf.disable())
            .cors(Customizer.withDefaults())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(
                    HttpMethod.OPTIONS, "/**"
                ).permitAll()
                .requestMatchers(
                    HttpMethod.GET, "/api/public"
                ).permitAll()
                .requestMatchers(
                    "/api/admin/**"
                ).hasRole("ADMIN")
                .requestMatchers(
                    "/api/demandes/**"
                ).hasAnyRole("USER", "ADMIN")
                .anyRequest().authenticated()
            )
            .oauth2ResourceServer(oauth -> oauth.jwt(
                jwt -> jwt.jwtAuthenticationConverter(
                    jwtConverter()
                )
            ))
            .build();
    }

    @Bean
    JwtAuthenticationConverter jwtConverter() {

        JwtGrantedAuthoritiesConverter scopes =
            new JwtGrantedAuthoritiesConverter();

        JwtAuthenticationConverter converter =
            new JwtAuthenticationConverter();

        converter.setJwtGrantedAuthoritiesConverter(jwt -> {

            Collection<GrantedAuthority> result =
                new ArrayList<>(scopes.convert(jwt));

            Map<String, Object> access =
                jwt.getClaimAsMap("realm_access");

            if (access != null &&
                access.get("roles") instanceof List<?> roles) {

                for (Object role : roles) {
                    result.add(
                        new SimpleGrantedAuthority(
                            "ROLE_" + role
                        )
```

```
                    );
                }
            }

            return result;
        });

        return converter;
    }

    @Bean
    CorsConfigurationSource corsConfigurationSource() {

        CorsConfiguration config =
            new CorsConfiguration();

        config.setAllowedOrigins(
            List.of("http://localhost:5173")
        );

        config.setAllowedMethods(
            List.of("GET", "POST", "PATCH", "OPTIONS")
        );

        config.setAllowedHeaders(
            List.of("Authorization", "Content-Type")
        );

        UrlBasedCorsConfigurationSource source =
            new UrlBasedCorsConfigurationSource();

        source.registerCorsConfiguration("/**", config);

        return source;
    }
}
```

6.1 Explication de la configuration

SecurityFilterChain :
Définit les règles de sécurité pour les endpoints REST.
oauth2ResourceServer :
Configure Spring Boot comme serveur de ressources acceptant des Access Tokens JWT.
JwtAuthenticationConverter :
Convertit les rôles présents dans le JWT de Keycloak en autorités reconnues par Spring Security.
hasRole("ADMIN") :
Vérifie que l'utilisateur possède l'autorité `ROLE_ADMIN`.
CorsConfigurationSource :
Autorise les requêtes provenant du frontend React.

Attention

Spring Boot ne génère plus les Access Tokens et ne vérifie plus directement les mots de passe. Cette responsabilité est confiée à Keycloak. Le backend reste responsable de la validation des jetons et du contrôle des autorisations.

7 Création de l'API REST

Nous allons développer une API de gestion des demandes.
Chaque demande possède :

— un identifiant ;
— un propriétaire ;
— un objet ;
— un statut.

Fichier : `DemandeController.java`

```
package ma.formation.demandesapi;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicLong;

import org.springframework.http.HttpStatus;
import org.springframework.security.oauth2.jwt.Jwt;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.server.ResponseStatusException;

@RestController
public class DemandeController {

    public record Demande(
        Long id,
        String owner,
        String objet,
        String statut
    ) {}

    public record NouvelleDemande(
        String objet
    ) {}

    public record StatutDemande(
        String statut
    ) {}

    private final AtomicLong sequence =
        new AtomicLong(0);

    private final List<Demande> demandes =
        new CopyOnWriteArrayList<>();

    @GetMapping("/api/public")
    public Map<String, String> publicApi() {

        return Map.of(
            "message",
            "API publique accessible"
        );
    }

    @GetMapping("/api/demandes")
    public List<Demande> mesDemandes(
            @AuthenticationPrincipal Jwt jwt) {

        return demandes.stream()
            .filter(d ->
                d.owner().equals(jwt.getSubject())
            )
            .toList();
```

```
    }

    @PostMapping("/api/demandes")
    public Demande creer(
            @AuthenticationPrincipal Jwt jwt,
            @RequestBody NouvelleDemande nouvelle) {

        if (nouvelle == null ||
            nouvelle.objet() == null ||
            nouvelle.objet().isBlank()) {

            throw new ResponseStatusException(
                HttpStatus.BAD_REQUEST,
                "Objet obligatoire"
            );
        }

        Demande demande = new Demande(
            sequence.incrementAndGet(),
            jwt.getSubject(),
            nouvelle.objet().trim(),
            "EN_ATTENTE"
        );

        demandes.add(demande);

        return demande;
    }

    @GetMapping("/api/admin/demandes")
    public List<Demande> toutesLesDemandes() {

        return List.copyOf(demandes);
    }

    @PatchMapping("/api/admin/demandes/{id}")
    public Demande changerStatut(
            @PathVariable Long id,
            @RequestBody StatutDemande body) {

        if (body == null ||
            !List.of(
                "EN_ATTENTE",
                "VALIDEE",
                "REFUSEE"
            ).contains(body.statut())) {

            throw new ResponseStatusException(
                HttpStatus.BAD_REQUEST,
                "Statut invalide"
            );
        }

        for (Demande d : new ArrayList<>(demandes)) {

            if (d.id().equals(id)) {

                Demande modifiee = new Demande(
                    d.id(),
                    d.owner(),
                    d.objet(),
                    body.statut()
                );

                demandes.remove(d);
```

```
                demandes.add(modifiee);

                return modifiee;
            }
        }

        throw new ResponseStatusException(
            HttpStatus.NOT_FOUND,
            "Demande introuvable"
        );
    }
}
```

7.1 Comprendre le JWT dans le contrôleur

L'annotation :

```
@AuthenticationPrincipal Jwt jwt
```

permet de récupérer le JWT de l'utilisateur authentifié.
La méthode :

```
jwt.getSubject()
```

récupère la valeur du claim `sub`, qui identifie l'utilisateur.
Cela permet d'associer automatiquement une demande à son propriétaire sans faire confiance à un identifiant fourni par React.

8 Démarrage du backend

Dans le dossier du projet Spring Boot :

```
./mvnw spring-boot:run
```

Sur Windows PowerShell :

```
.\mvnw.cmd spring-boot:run
```

Tester dans le navigateur :

http://localhost:8081/api/public

Résultat attendu :

```
{
  "message": "API publique accessible"
}
```

Tester ensuite :

http://localhost:8081/api/demandes

Sans Access Token, Spring Security doit répondre :

```
HTTP 401 Unauthorized
```

9 Séance 2 : React et OpenID Connect

9.1 Création du projet React

Depuis le dossier `atelier-oauth2`, exécuter :

```
npm create vite@latest demandes-front -- --template react
```

Entrer dans le projet :

```
cd demandes-front
```

Installer les dépendances :

```
npm install
npm install keycloak-js
```

9.2 Configuration de Vite

Fichier : `vite.config.js`

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    strictPort: true
  }
});
```

10 Configuration de Keycloak dans React

Créer le fichier suivant.

Fichier : `src/keycloak.js`

```
import Keycloak from 'keycloak-js';

const keycloak = new Keycloak({
  url: 'http://localhost:8080',
  realm: 'enset-realm',
  clientId: 'react-client'
});

export default keycloak;
```

Explication :
L'objet Keycloak contient les informations nécessaires pour communiquer avec le serveur d'identité :

— l'adresse de Keycloak ;

— le Realm ;

— l'identifiant du client React.

10.1 Initialisation de l'authentification

Fichier : `src/main.jsx`

```
import React from 'react';
import ReactDOM from 'react-dom/client';

import App from './App.jsx';
import keycloak from './keycloak.js';

keycloak.init({
  onLoad: 'login-required',
  pkceMethod: 'S256',
  checkLoginIframe: false
}).then(() => {

  ReactDOM.createRoot(
    document.getElementById('root')
  ).render(
    <React.StrictMode>
      <App />
    </React.StrictMode>
  );

}).catch((error) => {

  console.error(
    'Erreur de connexion Keycloak',
    error
  );

  document.getElementById('root').textContent =
    'Impossible de se connecter au serveur Keycloak.';
});
```

10.2 Explication

login-required :
L'utilisateur doit être authentifié pour accéder à l'application.
Dans le cas contraire, il est redirigé vers Keycloak.
pkceMethod : S256 :
Active PKCE avec SHA-256 pour sécuriser l'échange du code d'autorisation.
checkLoginIframe : false :
Désactive la vérification périodique de session par iframe pour simplifier l'exécution locale du TP.

11 Appels REST sécurisés depuis React

Créer un module permettant d'appeler les API Spring Boot avec un Access Token.

Fichier : `src/api.js`

```
import keycloak from './keycloak.js';

const API = 'http://localhost:8081';

export async function appelerApi(path, options = {}) {

  await keycloak.updateToken(30);

  const response = await fetch(API + path, {
```

```
    ...options,

    headers: {
      'Content-Type': 'application/json',

      Authorization: `Bearer ${keycloak.token}`,

      ...options.headers
    }
  });

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }

  return response.json();
}
```

11.1 Explication du code

La méthode :

```
await keycloak.updateToken(30);
```

demande le renouvellement du jeton si sa durée de validité restante est inférieure à 30 secondes.
L'instruction :

```
Authorization: `Bearer ${keycloak.token}`
```

transmet l'Access Token à Spring Boot.
Spring Security pourra alors vérifier la signature et les autorisations.

Attention

Les jetons doivent être manipulés avec précaution. Pour cet atelier, Keycloak JS les conserve en mémoire. Éviter de les copier dans localStorage.

12 Développement de l'interface React

Nous allons développer une interface permettant :

- d'afficher l'utilisateur connecté ;
- de créer une demande ;
- de consulter ses demandes ;
- de consulter toutes les demandes pour ADMIN ;
- de valider une demande pour ADMIN ;
- de se déconnecter.

Fichier : `src/App.jsx`

```
import { useEffect, useState } from 'react';

import keycloak from './keycloak.js';
import { appelerApi } from './api.js';

import './App.css';

export default function App() {
```

```
const [objet, setObjet] = useState('');

const [mesDemandes, setMesDemandes] =
  useState([]);

const [toutes, setToutes] =
  useState([]);

const [message, setMessage] =
  useState('');

const roles =
  keycloak.tokenParsed?.realm_access?.roles || [];

const isAdmin =
  roles.includes('ADMIN');

const username =
  keycloak.tokenParsed?.preferred_username
  || 'Utilisateur';

async function charger() {

  try {

    setMesDemandes(
      await appelerApi('/api/demandes')
    );

    if (isAdmin) {

      setToutes(
        await appelerApi('/api/admin/demandes')
      );
    }

  } catch (e) {

    setMessage('Erreur : ' + e.message);
  }
}

useEffect(() => {
  charger();
}, []);

async function creer(e) {

  e.preventDefault();

  try {

    await appelerApi('/api/demandes', {
      method: 'POST',
      body: JSON.stringify({
        objet: objet
      })
    });

    setObjet('');

    setMessage('Demande creee.');

    await charger();
```

```
  } catch (e) {

    setMessage('Erreur : ' + e.message);
  }
}

async function valider(id) {

  try {

    await appelerApi(
      `/api/admin/demandes/${id}`,
      {
        method: 'PATCH',

        body: JSON.stringify({
          statut: 'VALIDEE'
        })
      }
    );

    setMessage('Demande validee.');

    await charger();

  } catch (e) {

    setMessage('Erreur : ' + e.message);
  }
}

return (

  <main>

    <header>

      <h1>Gestion des demandes</h1>

      <p>
        Connecte :
        <strong> {username}</strong>
      </p>

      <p>
        Roles : {roles.join(', ')}
      </p>

      <button onClick={() =>
        keycloak.logout({
          redirectUri: 'http://localhost:5173/'
        })
      }>
        Se deconnecter
      </button>

    </header>

    <section>

      <h2>Creer une demande</h2>

      <form onSubmit={creer}>

        <input
```

```
          value={objet}
          onChange={e =>
            setObjet(e.target.value)
          }
          placeholder="Objet de la demande"
          required
        />

        <button type="submit">
          Envoyer
        </button>

      </form>

    </section>

    <section>

      <h2>Mes demandes</h2>

      <ul>

        {mesDemandes.map(d => (

          <li key={d.id}>

            #{d.id} - {d.objet}
            ({d.statut})

          </li>

        ))}

      </ul>

    </section>

    {isAdmin && (

      <section>

        <h2>
          Administration : toutes les demandes
        </h2>

        <ul>

          {toutes.map(d => (

            <li key={d.id}>

              #{d.id} - {d.objet}
              ({d.statut})

              {d.statut !== 'VALIDEE' && (

                <button
                  onClick={() => valider(d.id)}
                >
                  Valider
                </button>

              )}

            </li>
```

```
            ))}

          </ul>

        </section>

      )}

      <p role="status">
        {message}
      </p>

    </main>
  );
}
```

12.1 Style de l'application

Fichier : `src/App.css`

```
body {
  font-family: Arial, sans-serif;
  background: #f2f5fa;
}

main {
  max-width: 850px;
  margin: 25px auto;
}

header, section {
  background: white;
  padding: 20px;
  margin: 15px 0;
  border-radius: 10px;
  box-shadow: 0 1px 5px #ccc;
}

button {
  background: #1768ac;
  color: white;
  border: 0;
  border-radius: 5px;
  padding: 9px 13px;
  margin: 5px;
  cursor: pointer;
}

input {
  padding: 9px;
  min-width: 260px;
}

li {
  margin: 8px 0;
}
```

13 Démarrage de React

Dans le dossier React, exécuter :

```
npm run dev
```

Ouvrir :

http://localhost:5173

Le navigateur doit rediriger automatiquement l'utilisateur vers la page de connexion Keycloak.

14 Tests fonctionnels et de sécurité

14.1 Test 1 : Authentification de Alice

Se connecter avec :

— Username : `alice`

— Password : `alice123`

Vérifier les résultats suivants :

1. L'utilisateur accède à React.
2. Son nom est affiché.
3. Il peut créer une demande.
4. Il peut consulter ses demandes.
5. Le panneau Administration est absent.

14.2 Test 2 : Authentification de Bob

Se déconnecter puis se connecter avec :

— Username : `bob`

— Password : `bob123`

Vérifier :

1. Bob accède à l'application.
2. Il peut consulter les demandes.
3. Le panneau Administration est visible.
4. Il peut consulter toutes les demandes.
5. Il peut valider une demande.

14.3 Test 3 : Vérification des Access Tokens

Ouvrir les outils de développement du navigateur.
Dans l'onglet **Network**, observer une requête vers :

```
GET http://localhost:8081/api/demandes
```

Observer l'en-tête :

```
Authorization: Bearer eyJ...
```

Le jeton est transmis à Spring Boot, qui vérifie son authenticité et les autorisations.

14.4 Test 4 : API sans Access Token

Dans Postman, exécuter :

```
GET http://localhost:8081/api/demandes
```

Ne pas ajouter de token.
Résultat attendu :

```
HTTP 401 Unauthorized
```

14.5 Test 5 : Accès ADMIN avec Alice

Dans Postman, utiliser le JWT de Alice et exécuter :

```
GET http://localhost:8081/api/admin/demandes
```

Ajouter dans l'onglet Authorization :

```
Type: Bearer Token
Token: ACCESS_TOKEN_ALICE
```

Résultat attendu :

```
HTTP 403 Forbidden
```

14.6 Test 6 : Accès ADMIN avec Bob

Reprendre la même requête avec l'Access Token de Bob.
Résultat attendu :

```
HTTP 200 OK
```

14.7 Test 7 : Jeton invalide

Modifier un caractère de l'Access Token avant de l'envoyer dans Postman.
Résultat attendu :

```
HTTP 401 Unauthorized
```

Attention

Le fait de masquer le panneau ADMIN dans React n'est pas une mesure de sécurité suffisante. La véritable protection est assurée dans le backend par Spring Security.

15 Analyse du contenu d'un JWT

Examiner les principaux claims présents dans l'Access Token délivré par Keycloak.

sub
Identifiant unique de l'utilisateur.

iss
Adresse de l'émetteur du JWT.

exp
Date d'expiration du jeton.

iat
Date de création du jeton.

realm_access.roles
Rôles associés à l'utilisateur.

scope
Scopes OAuth 2.0 associés au jeton.

Il est important de distinguer les scopes OAuth 2.0 des rôles applicatifs.

Attention

Ne jamais publier un Access Token réel. Un jeton actif peut permettre l'accès aux ressources protégées.

16 Travail supplémentaire

À partir de l'application développée, réaliser les améliorations suivantes.

16.1 Fonctionnalité 1 : Refuser une demande

Ajouter un bouton **Refuser** dans l'espace administrateur.
Le bouton doit envoyer une requête PATCH pour modifier le statut de la demande à :

```
REFUSEE
```

Cette fonctionnalité doit être réservée aux utilisateurs possédant le rôle ADMIN.

16.2 Fonctionnalité 2 : Profil utilisateur

Ajouter un endpoint :

```
GET /api/profil
```

Cet endpoint doit retourner :

— l'identifiant de l'utilisateur ;
— son nom d'utilisateur.

Les données doivent être extraites du JWT de l'utilisateur authentifié.

16.3 Fonctionnalité 3 : Tests de sécurité

Vérifier les comportements suivants :

1. Une requête sans token est refusée.
2. Une requête avec un token invalide est refusée.
3. Un utilisateur USER ne peut pas modifier une demande via l'API d'administration.
4. Un utilisateur ADMIN peut modifier le statut.

17 Bilan de l'atelier

Dans cette application :

— **Keycloak** joue le rôle d'Authorization Server et d'OpenID Provider.
— **React** joue le rôle de client OAuth 2.0 / OpenID Connect.
— **Spring Boot** joue le rôle de Resource Server.
— **Spring Security** valide les JWT et applique les règles d'autorisation.

Contrairement à une authentification JWT développée directement avec Spring Boot, les utilisateurs s'authentifient désormais auprès de Keycloak.

Cette architecture facilite la centralisation de l'authentification et peut être étendue à plusieurs applications et microservices.